Pytest Loguru

Loguru **1**
Installation et import 1
Utilisation simple 1
Sauvegarde dans un fichier de log 2
Configuration 2
Configuration des fichiers 4
Rotation 4
retention 4
compression 5
enqueue 5
Exemple complet (login d'un api) 5
Pytest pytest documentation **7**
Installation et utilisation 7
Architecture 7
Test unitaire simple 8
Tests multiples avec parametrize 8
Les fixtures pour préparer des données 9
exemple avec une fixture de base de données SQLite 9
Test de route d'une API: 10

Loguru

Loguru est une librairie Python qui rend la journalisation (logging) simple, agréable et puissante. Contrairement au module `logging` standard, Loguru offre :

- Une configuration ultra-simple
- Une sortie colorée par défaut dans le terminal
- Des infos contextuelles (fichier, ligne, etc.)
- Une gestion facile des fichiers de logs (rotation, rétention, compression)
- Un seul logger préconfiguré : `logger`

Installation et import

Pour installer Loguru, utilisez la commande pip suivante dans votre terminal ou intégrer le dans votre requirements.txt:

```
pip install loguru
```

Pour importer Loguru dans votre code Python, utilisez simplement :

```
from loguru import logger
```

Utilisation simple

```
from loguru import logger
# Niveaux de log standard (par ordre de sévérité)
# TRACE, DEBUG, INFO, WARNING, ERROR, CRITICAL
logger.trace("Ceci est un message de trace.")
logger.debug("Informations de débogage.")
logger.info("Informations générales.")
logger.warning("Attention")
```

```
logger.error("Une erreur s'est produite.")
logger.critical("Erreur critique !")
```

Ces différents niveaux de log donne les résultats suivant:

```
2025-04-11 11:07:13.972 | DEBUG    | __main__:<module>:7 - Informations de débogage.
2025-04-11 11:07:13.974 | INFO     | __main__:<module>:8 - Informations générales.
2025-04-11 11:07:13.975 | WARNING  | __main__:<module>:9 - Attention
2025-04-11 11:07:13.975 | ERROR    | __main__:<module>:10 - Une erreur s'est produite.
2025-04-11 11:07:13.975 | CRITICAL | __main__:<module>:11 - Erreur critique !
```

Sauvegarde dans un fichier de log

Configuration

Afin de sauvegarder les informations dans un fichier de log, il faut configurer votre logger :

```
from loguru import logger


# Fichier pour les erreurs et les niveaux critiques
logger.add("errors.log", level="ERROR",
format="{time} {level} {message}")
# Fichier pour les avertissements
logger.add("warnings.log", level="WARNING",
format="{time} {level} {message}")
# Fichier pour les informations générales
logger.add("info.log", level="INFO", format="{time}
{level} {message}")
# Un fichier pour tout, du débogage au critique (si
vous le souhaitez)
logger.add("all.log", level="DEBUG", format="{time}
{level} {message}")
```

```
logger.debug("Ceci est un message de débogage.")
logger.info("Informations importantes.")
logger.warning("Attention !")
logger.error("Une erreur s'est produite.")
logger.critical("Erreur critique !")
```

Loguru enregistre tout ce qui est du niveau spécifié *et supérieur*.

TRACE	**TRACE**, **DEBUG**, **INFO**, **WARNING**, **ERROR**, **CRITICAL**
DEBUG	**DEBUG**, **INFO**, **WARNING**, **ERROR**, **CRITICAL**
INFO	**INFO**, **WARNING**, **ERROR**, **CRITICAL**
WARNING	**WARNING**, **ERROR**, **CRITICAL**
ERROR	**ERROR**, **CRITICAL**
CRITICAL	**CRITICAL**

Exemple du fichier warnings.log

```
2025-04-11T11:13:19.786676+0200 WARNING Attention !
2025-04-11T11:13:19.787362+0200 ERROR Une erreur s'est produite.
2025-04-11T11:13:19.788241+0200 CRITICAL Erreur critique !
```

Configuration des fichiers

Loguru permet de gérer automatiquement la taille, la durée de vie et la compression des fichiers de log grâce à quelques options très simples. Ainsi on peut décider de la taille maximum des fichiers, du taux de compression ou combien de temps on veut garder le fichier.

```
logger.add(
    # Fichier avec timestamp
    "logs/log_{time}.log",
    # Nouveau fichier tous les 10 Mo
    rotation="10 MB",
    # Conserver les logs 7 jours
    retention="7 days",
    # Compresser les anciens logs
    compression="zip",
    # Niveau minimal
    level="DEBUG",
    format="{time} {level} {message}"
)
```

Rotation

Quand créer un nouveau fichier.

- "10 MB" → à partir de 10 Mo
- "1 week" → chaque semaine
- "00:00" → chaque jour à minuit

retention

Combien de temps ou combien de fichiers garder.

- "7 days" → garde les fichiers 7 jours
- 5 → garde les 5 derniers fichiers

Loguru conserve uniquement les 5 derniers fichiers générés par rotation. Dès qu'un 6e fichier est créé (suite à une nouvelle rotation), le plus ancien est automatiquement supprimé.

compression

Compression des anciens fichiers dans différents format : "zip", "gz", "bz2"

enqueue

Sécurise l'écriture en contexte multi-thread/process (recommandé en cas de logs depuis plusieurs threads/process)

Exemple complet (login d'un api)

```
from fastapi import FastAPI, HTTPException, Form
from loguru import logger

app = FastAPI()

# === CONFIGURATION LOGURU ===

logger.remove()  # Nettoie la config de base

# --- CONFIG DEV ---
logger.add("logs/dev_{time}.log",
          rotation="10 MB",
          retention="7 days",
          compression="zip",
          level="TRACE",
          enqueue=True,
          format="{time:YYYY-MM-DD HH:mm:ss} | {level} | {message}")

# --- CONFIG PROD (commentée) ---
# logger.add("/var/log/app/app.log",
#            rotation="100 MB",
#            retention="30 days",
#            compression="gz",
#            level="INFO",
#            enqueue=True,
#            format="{time:YYYY-MM-DD HH:mm:ss} | {level} | {message}")
```

```python
# === ROUTE LOGIN ===

@app.post("/login")
def login(username: str = Form(...), password: str = Form(...)):
    logger.trace("Requête de login reçue")

    logger.debug(f"Tentative de login pour l'utilisateur : {username}")

    try:
        # Simulation de vérification
        if not username or not password:
            logger.warning("Identifiants manquants")
            raise HTTPException(status_code=400, detail="Identifiants requis")

        # Simuler un utilisateur en base
        if username != "admin":
            logger.info(f"Utilisateur inconnu : {username}")
            raise HTTPException(status_code=401, detail="Utilisateur non trouvé")

        if password != "secret":
            logger.warning(f"Mot de passe incorrect pour {username}")
            raise HTTPException(status_code=403, detail="Mot de passe invalide")

        logger.info(f"Connexion réussie pour {username}")
        return {"message": f"Bienvenue, {username}!"}

    except HTTPException as http_exc:
        logger.error(f"Erreur de login pour {username} : {http_exc.detail}")
        raise

    except Exception as e:
        logger.critical(f"Erreur inattendue lors du login : {e}")
        logger.exception("Détail de l'exception")
        raise HTTPException(status_code=500, detail="Erreur serveur")
```

Pytest pytest documentation

Pytest est un framework de test pour Python à la fois simple et puissant. Il permet d'écrire rapidement des tests unitaires, des tests d'API, et de simuler ("mocker") des objets comme les bases de données. Il fonctionne sans configuration spéciale et reconnaît automatiquement les fichiers et les fonctions de test contenues dans les fichiers.

Installation et utilisation

Pour installer pytest il suffit d'utiliser pip ou de l'inclure dans le requirements.txt:

```
pip install pytest
```

Il n'est pas toujours nécessaire de l'importer.

Pour exécuter les tests il suffit de lancer :

```
pytest
```

dans un terminal, on peut aussi spécifier le dossier ou un des fichiers en particulier:

```
pytest tests/test_math.py
```

Architecture

Pour bien organiser son code, on range les fichiers de tests dans un dossier tests:

```
tests/
├── test_math.py
├── test_login.py
└── conftest.py
```

Test unitaire simple

Pytest exécute automatiquement les fonctions commençant par `test_` dans les fichiers `test_*.py`.

par exemple, le fichier : `test_math.py`

```
def add(a, b):
    return a + b


def test_add():
    assert add(2, 3) == 5
```

Il faut noter que la fonction add aurait pu être située dans un autre dossier (modules par exemple) et être importée dans le fichier de test.

Tests multiples avec parametrize

On peut effectuer des séries de tests de la même fonction de test d'un seul coup:

```
import pytest
@pytest.mark.parametrize("a, b, result", [
    (2, 3, 6),
    (0, 10, 0),
    (-1, 5, -5),
])
def test_multiply(a, b, result):
    assert multiply(a, b) == result
```

```
collected 3 items
tests/test_multiply.py ...                                [100%]
================= 3 passed in 0.01s =================
```

Les fixtures pour préparer des données

Une fixture rend accessible aux fonctions des paramètres prédéfinis.

```
import pytest

@pytest.fixture
def user():
    return {"username": "admin", "password": "secret"}

def test_user_data(user):
    assert user["username"] == "admin"
```

La fixture `user` est nécessaire à l'exécution du test : `test_user_data`
Ici,il s'agit d'un dictionnaire mais cela aurait pu être une donnée issue d'une base de données, d'un fichier ou un objet comme un connecteur etc...

exemple avec une fixture de base de données SQLite

```
import sqlite3
import pytest

@pytest.fixture

def sqlite_db():
    conn = sqlite3.connect(":memory:")
    cursor = conn.cursor()
    cursor.execute("CREATE TABLE …")
    cursor.execute("INSERT INTO users … ")
    conn.commit()
    yield conn
    conn.close()



def get_user_by_username(conn, username):
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM users …", (username,))

    return cursor.fetchone()
```

Test de route d'une API:

```
from fastapi.testclient import TestClient
from main import app

client = TestClient(app)

def test_root_route():
    response = client.get("/")
    assert response.status_code == 200
    assert response.json()["message"] == "Bienvenue
sur l'API"

def test_faq_route():
    response = client.get("/FAQ")
    assert response.status_code == 200
    assert "faq" in response.json()
```

de l'API:

```
from fastapi import FastAPI, Form, HTTPException

app = FastAPI()

@app.get("/")
def root():
    return {"message": "Bienvenue sur l'API"}

@app.get("/FAQ")
def faq():
    return {"faq": "Voici la foire aux questions."}
```